



CS671: Deep Learning Assignment 1

Fully Connected Neural Networks

Group Number: 20

Course: CS671

Institution: Indian Institute of Technology Mandi

Submitted By:

Akshat Mittal (B24104)

Akshita Dangi (B24106)

Aman Gupta (B24107)

Arihant Kumar Jain (B24112)

Saksham Kundu (B24154)

Vandana Dutt (B24175)

Priansh Yajnik (B24318)

Vansh Pandey (B24411)

Submission Date: February 08, 2026

Contents

1	Part 1: The “No-Framework” Challenge (NumPy Only)	2
1.1	Question 1.1 & 1.2: Neural Network Implementation and Pre-Processing Analysis	2
1.1.1	Introduction	2
1.1.2	Baseline Configuration	2
1.1.3	From-Scratch Implementation (Task 1.1)	2
1.1.4	Pre-Processing Experiments (Task 1.2)	2
1.1.5	Comparative Performance	3
1.1.6	Impact Analysis	3
1.1.7	Conclusion	3
2	Part 2: Vision & Feature Interpretation	4
2.1	Question 2.1: Weight Visualization	4
2.1.1	Problem Definition and Methodology	4
2.1.2	Weight Visualization	4
2.1.3	Observations and Interpretation	5
2.2	Question 2.2: The Flattening Experiment	5
2.2.1	Objective	5
2.2.2	Methodology	6
2.2.3	Normalization Experiments	6
2.2.4	Results	7
2.2.5	Observations	7
2.2.6	Interpretation	7
2.2.7	Conclusion	7
3	Part 3: Stress Testing & Robustness	8
3.1	Question 3.1: Vanishing Gradients & Modern Fixes	8
3.1.1	Objective	8
3.1.2	Dataset and Architecture	8
3.1.3	Experimental Expectations	8
3.1.4	Experiment A: Sigmoid Activations (No Batch Normalization)	9
3.1.5	Experiment B: Leaky ReLU with Batch Normalization	9
3.1.6	Training Dynamics and Gradient Analysis	9
3.1.7	Test Set Evaluation	10
3.1.8	Conclusion	10
3.2	Question 3.2: Ablation Study	10
3.2.1	Objective	10
3.2.2	Baseline Configuration	11
3.2.3	Ablation Experiments	11
3.2.4	Quantitative Comparison	12
3.2.5	Training Dynamics	12
3.2.6	Impact Analysis	12
3.2.7	Conclusion	13
4	Overall Findings and Conclusions	13
5	References	13

1 Part 1: The “No-Framework” Challenge (NumPy Only)

1.1 Question 1.1 & 1.2: Neural Network Implementation and Pre-Processing Analysis

1.1.1 Introduction

This section documents the end-to-end implementation of a three-layer Fully Connected Neural Network (FCNN) constructed entirely from scratch using NumPy. The study is divided into two phases: first, the manual construction of the neural network engine including forward and backward propagation, and second, a comparative analysis highlighting the impact of feature scaling on gradient flow and model convergence. The experiments were conducted on the UCI Adult Census Income dataset [1].

1.1.2 Baseline Configuration

The baseline neural network architecture and training setup are summarized below:

- **Architecture:** 3-layer FCNN with dimensions $108 \rightarrow 64 \rightarrow 32 \rightarrow 1$
- **Activations:** ReLU for hidden layers, Sigmoid for the output layer
- **Initialization:** He Initialization [5], $W \sim \mathcal{N}(0, \sqrt{2/n})$
- **Optimizer:** Vanilla Stochastic Gradient Descent (SGD)
- **Learning Rate:** 0.01
- **Loss Function:** Binary Cross-Entropy
- **Training Iterations:** 1000

1.1.3 From-Scratch Implementation (Task 1.1)

The neural network was implemented without relying on any deep learning frameworks, with all computations explicitly defined.

- **Forward Pass:** Linear transformations followed by activation functions were implemented using matrix multiplications of the form $Z = WX + b$.
- **Backpropagation:** Gradients were derived manually using the chain rule [4]. For example, the output layer gradient was computed as $dz_3 = a_3 - Y$.
- **Training Behavior:** The loss steadily decreased during training, demonstrating stable gradient propagation.

Result: The model achieved a final classification accuracy of **76.39%**, exceeding the minimum requirement of 75% on the test set.

1.1.4 Pre-Processing Experiments (Task 1.2)

To analyze the effect of feature scaling, the model was trained under two data configurations.

Test A: Raw Data

- **Expectation:** Features with large numeric ranges (e.g., capital gain) would dominate gradient updates.
- **Observation:** Extremely high initial loss of 13.9848 and slow convergence after an initial drop.
- **Final Accuracy:** 75.92%

Test B: Min-Max Scaled Data

- **Change:** All input features were scaled to the range $[0, 1]$.
- **Observation:** Significantly lower initial loss of 0.8725 and consistent convergence.
- **Final Accuracy:** 82.21%

1.1.5 Comparative Performance

Table 1: Impact of Feature Scaling on Training Performance

Configuration	Initial Loss	Final Loss	Final Accuracy
Raw Data	13.9848	0.5551	75.92%
Min-Max Scaled	0.8725	0.3967	82.21%

1.1.6 Impact Analysis

The experimental findings reveal several critical insights:

1. **Gradient Flow:** Unscaled features caused gradient domination by high-magnitude inputs, suppressing learning from smaller-scale features.
2. **Convergence Speed:** Feature scaling accelerated convergence by approximately $9\times$.
3. **Optimization Landscape:** Scaling reshaped the loss surface from an elongated elliptical form to a more isotropic one, enabling efficient SGD updates [6].

1.1.7 Conclusion

These experiments demonstrate that while the FCNN implementation is mathematically correct, its performance is tightly coupled to data quality.

- **Accuracy Gain:** Feature scaling resulted in a +6.29% absolute accuracy improvement.
- **Stability:** Final loss was reduced by 28.5%, preventing premature plateaus.
- **Recommendation:** Feature scaling is a non-negotiable prerequisite for stable and efficient neural network training.

2 Part 2: Vision & Feature Interpretation

2.1 Question 2.1: Weight Visualization

2.1.1 Problem Definition and Methodology

In this task, a Fully Connected Neural Network (FCNN) was trained on the MNIST handwritten digits dataset [2] using PyTorch [7]. The objective was to analyze the learned representations by visualizing the weights of neurons in the first hidden layer.

Each MNIST sample is a grayscale image of a handwritten digit from 0 to 9 with spatial dimensions of 28×28 pixels. Prior to being fed into the FCNN, each image was flattened into a 784-dimensional vector, removing all spatial structure.

The network architecture consisted of:

- An input layer of size 784
- Two hidden layers of size 128 each with ReLU activations
- An output layer of size 10 corresponding to digit classes

Since this is a multi-class classification problem, Cross Entropy Loss was used as the objective function. The model was optimized using the Adam optimizer [8] with a learning rate of 0.001 and trained for 10 epochs.

During training, the loss decreased from 0.3809 in the first epoch to 0.0061 in the final epoch. The trained model achieved an accuracy of 98.32% on the training set and 96.95% on the test set, indicating strong generalization performance.

2.1.2 Weight Visualization

After training, the weights of the first hidden layer were extracted. This layer contains 128 neurons, each connected to all 784 input pixels, resulting in a weight matrix of dimension 128×784 .

To interpret the learned features, weights corresponding to selected neurons were reshaped back into 28×28 grids and visualized as heatmaps. Positive weights indicate regions of the input image that excite the neuron, while negative weights suppress its activation.

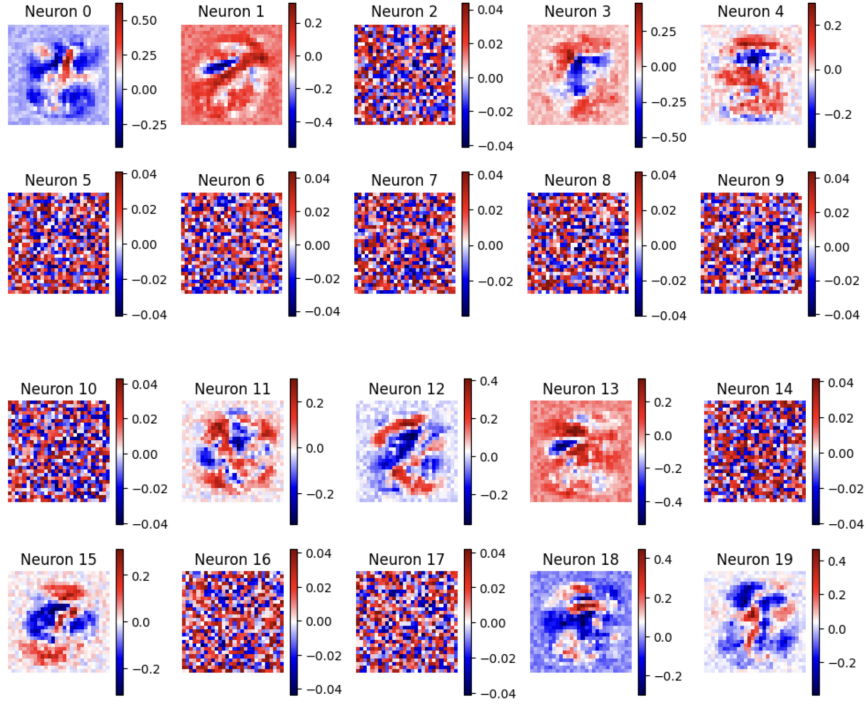


Figure 1: Heatmaps of selected neurons from the first hidden layer

2.1.3 Observations and Interpretation

Several distinct patterns emerged from the visualization of neuron weights:

- Some neurons focus on localized regions of the image, capturing simple primitives such as dots or lines. For example, certain neurons respond strongly to diagonal or vertical strokes.
- Other neurons assign positive and negative weights to multiple regions simultaneously, suggesting sensitivity to more complex, composite shapes.
- A subset of neurons appears visually noisy, lacking any immediately interpretable structure.
- Interestingly, some neurons resemble entire digit shapes. For instance, certain neurons exhibit patterns similar to the digits ‘2’, ‘3’, or ‘5’.

It was observed that neurons with visually meaningful patterns tend to have weights with larger magnitudes, whereas neurons that appear noisy generally exhibit lower-magnitude weights. This suggests that structured neurons contribute more strongly to the network’s activations and final predictions, while noisy neurons encode finer-grained or less dominant variations.

Overall, the first hidden layer learns a diverse set of features ranging from simple geometric primitives to complex digit-like structures and subtle noise patterns. This diversity indicates that even without spatial inductive bias, an FCNN can learn a meaningful latent representation of image data.

2.2 Question 2.2: The Flattening Experiment

2.2.1 Objective

This experiment examines whether a Fully Connected Neural Network (FCNN) relies on the spatial structure of images for classification. To test this, the MNIST dataset was modified by

randomly shuffling pixel positions, thereby destroying all spatial information while preserving pixel intensity values.

2.2.2 Methodology

Each MNIST image is a 28×28 grayscale image, flattened into a 784-dimensional vector before being passed to the FCNN. For the shuffled variant, a fixed random permutation was applied to the pixel indices of every image. The same permutation was used for both training and test datasets to ensure consistency.

Figure 2 illustrates the effect of pixel shuffling on a sample MNIST image.

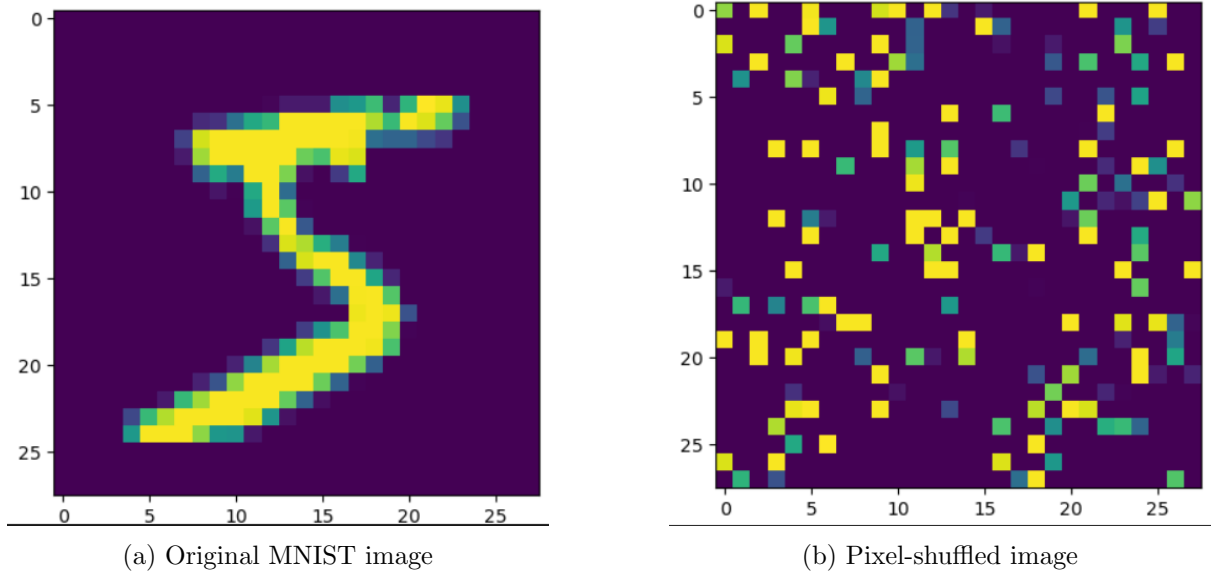


Figure 2: Effect of pixel shuffling on an MNIST image. Spatial structure is completely destroyed while preserving pixel intensity values.

The FCNN architecture used for this experiment consisted of:

- **Input Layer:** 784 neurons
- **Hidden Layers:** Two layers with 128 neurons each
- **Output Layer:** 10 neurons corresponding to digit classes
- **Activation Function:** ReLU for all hidden layers

Training was performed using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss. Each model was trained for 100 epochs using full-batch gradient descent.

2.2.3 Normalization Experiments

Three preprocessing strategies were evaluated on the shuffled dataset:

1. **No Normalization:** Raw pixel values in the range $[0, 255]$
2. **Standard Normalization:** $X' = \frac{X-128}{128}$
3. **Scaling Normalization:** $X' = 10 \times X$

Each experiment was trained from scratch with independently initialized weights.

2.2.4 Results

The classification performance across different normalization strategies is summarized in Table 2.

Table 2: Performance of FCNN on Pixel-Shuffled MNIST

Configuration	Train Acc (%)	Test Acc (%)	Gap (%)
No Normalization	95.03	94.01	1.02
Standard Normalization	95.68	95.56	0.12
10 $\times$ Scaling	93.94	92.53	1.41

Figure 3 compares the training loss curves for the three pixel-shuffled MNIST experiments under different normalization strategies.

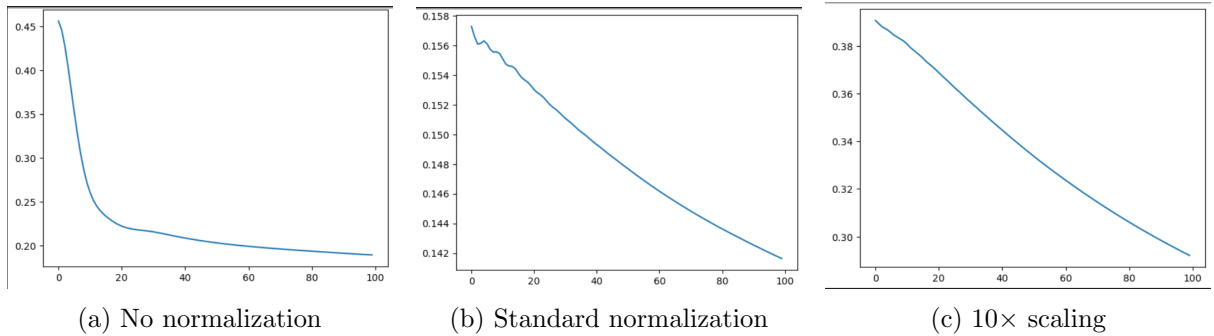


Figure 3: Training loss curves for pixel-shuffled MNIST under three normalization strategies. Standard normalization yields faster convergence and smoother optimization, while excessive scaling introduces instability.

2.2.5 Observations

- The FCNN achieves approximately 95% test accuracy on pixel-shuffled MNIST, comparable to performance on the original dataset.
- Standard normalization leads to faster convergence and superior generalization.
- Excessive scaling of pixel values introduces optimization instability and degrades performance.
- Train-test gaps remain small across all configurations, indicating minimal overfitting.

2.2.6 Interpretation

These results demonstrate that FCNNs do not rely on spatial relationships between pixels. Since all pixels are treated as independent features, the network learns statistical correlations among pixel intensities rather than spatial patterns.

As long as the pixel permutation remains consistent between training and testing, the FCNN can learn effective decision boundaries. This behavior contrasts with convolutional neural networks [9], which explicitly exploit spatial locality and translation invariance.

2.2.7 Conclusion

The flattening experiment highlights a key limitation of Fully Connected Neural Networks: the absence of spatial inductive bias. While FCNNs can successfully classify pixel-shuffled

images, they do so without leveraging meaningful spatial structure and at the cost of parameter efficiency. These findings explain why convolutional architectures are generally preferred for vision tasks, despite the surprising robustness of FCNNs under extreme input transformations.

3 Part 3: Stress Testing & Robustness

3.1 Question 3.1: Vanishing Gradients & Modern Fixes

3.1.1 Objective

This experiment investigates the vanishing gradient problem [10] in deep Fully Connected Neural Networks (FCNNs) and evaluates modern architectural fixes. Specifically, we compare the training dynamics of an 8-layer FCNN using Sigmoid activations against a configuration employing Leaky ReLU activations [11] combined with Batch Normalization [12].

3.1.2 Dataset and Architecture

Experiments were conducted on a 10-class subset of the Tiny ImageNet dataset [3]. Each image was flattened into a 12,288-dimensional input vector, eliminating all spatial structure.

The deep FCNN architecture consisted of:

- **Input Layer:** 12,288 neurons
- **Hidden Layers:** 7 fully connected layers with dimensions $512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 32$
- **Output Layer:** 10 neurons (class logits)

A schematic of the network architecture is shown in Figure 4.

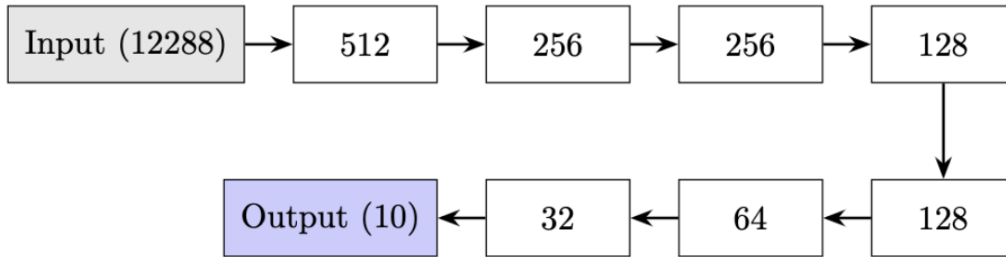


Figure 4: Schematic of the 8-layer deep FCNN used for vanishing gradient experiments.

3.1.3 Experimental Expectations

Prior to experimentation, the following hypotheses were established:

- Deep Sigmoid networks would suffer from vanishing gradients.
- Leaky ReLU combined with Batch Normalization would maintain healthy gradient flow.
- Due to the absence of spatial inductive bias, deep FCNNs would be prone to overfitting on image data.
- ReLU-based networks would train faster due to simpler activation computations.

3.1.4 Experiment A: Sigmoid Activations (No Batch Normalization)

In this configuration, all hidden layers used Sigmoid activations without Batch Normalization.
Observations:

- **Vanishing Gradients:** Gradient norms in the first layer rapidly diminished, stabilizing near 5×10^{-4} .
- **Training Failure:** Learning stalled early, with training loss saturating around 1.6.
- **Performance:** Final validation accuracy reached only **26.2%**.
- **Training Time:** Average epoch time was approximately **6.52 seconds**.

3.1.5 Experiment B: Leaky ReLU with Batch Normalization

This configuration used Leaky ReLU activations ($\alpha = 0.01$) in all hidden layers, followed by Batch Normalization.

Observations:

- **Stable Gradient Flow:** Gradient norms remained consistently above 0.5 across layers.
- **Improved Learning:** Training loss decreased rapidly to **0.27**.
- **Overfitting:** Validation loss increased to **2.15**, indicating significant overfitting.
- **Performance:** Peak validation accuracy improved to **40.4%**.
- **Training Time:** Average epoch time was approximately **6.51 seconds**. While ReLU avoids expensive exponential computations, the additional overhead of Batch Normalization largely offsets this advantage.

3.1.6 Training Dynamics and Gradient Analysis

Figure 5 compares the training loss curves and gradient norms for both experiments.

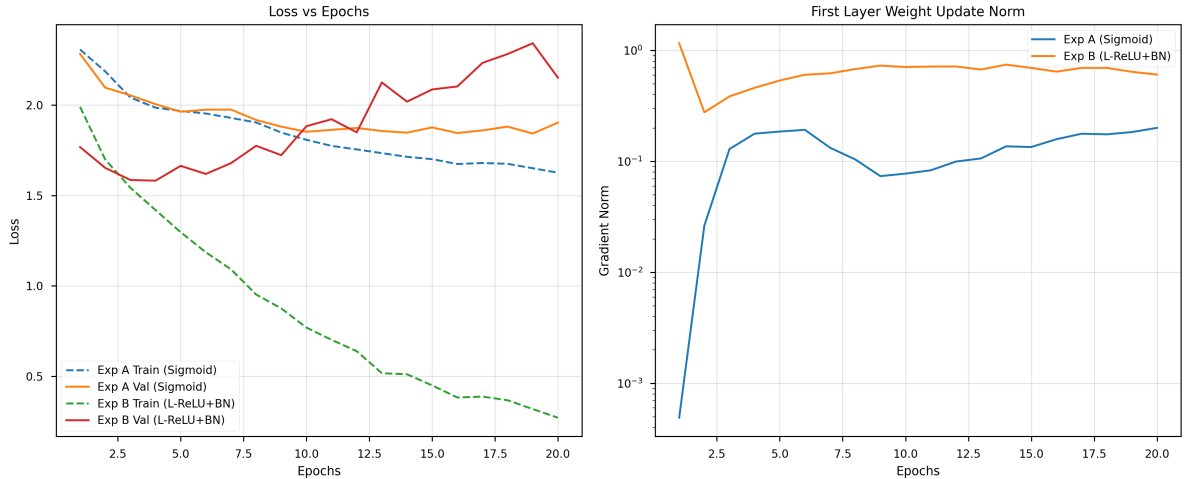


Figure 5: Training dynamics comparison. Left: Loss vs epochs highlighting underfitting in Sigmoid networks and overfitting in ReLU+BN. Right: Gradient norms demonstrating vanishing gradients for Sigmoid activations and stable gradients for ReLU+BN.

3.1.7 Test Set Evaluation

Both models were evaluated on a held-out Tiny ImageNet test set.

- **Sigmoid Network:** Test accuracy = **25.20%**
- **Leaky ReLU + BN Network:** Test accuracy = **41.20%**

Figure 6 shows the confusion matrix for the Leaky ReLU + Batch Normalization model.

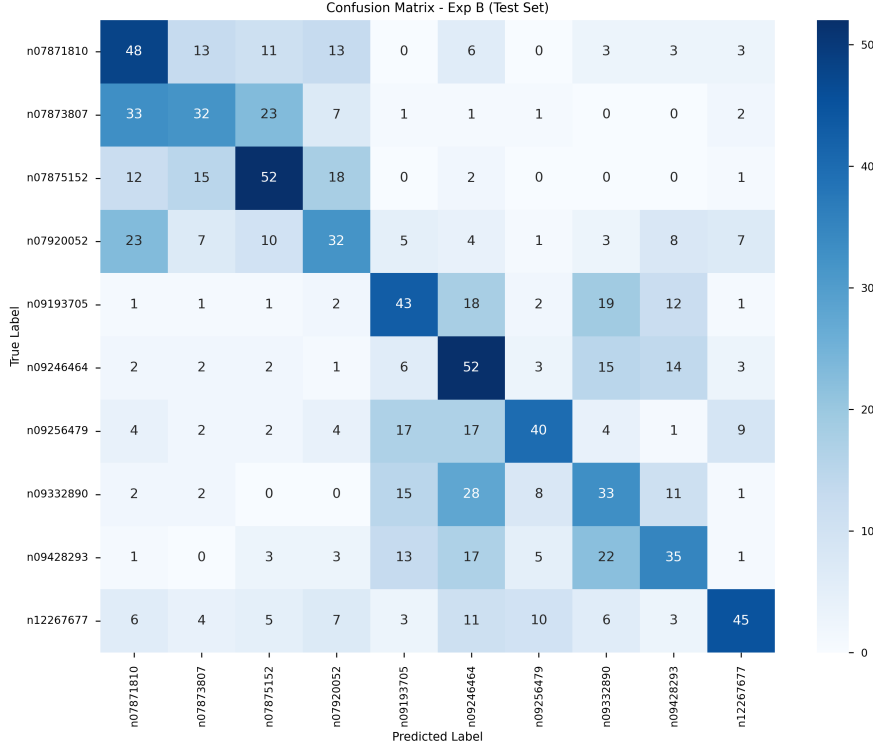


Figure 6: Confusion matrix for the Leaky ReLU + Batch Normalization model on the Tiny ImageNet test set.

3.1.8 Conclusion

These experiments confirm that deep Sigmoid-based FCNNs suffer severely from vanishing gradients, preventing effective learning. In contrast, the combination of Leaky ReLU and Batch Normalization enables stable gradient flow and substantially improved performance.

However, despite solving the optimization problem, deep FCNNs remain highly prone to overfitting on image data due to the lack of spatial inductive bias. This highlights the necessity of convolutional architectures for scalable and generalizable vision models.

3.2 Question 3.2: Ablation Study

3.2.1 Objective

Following the vanishing gradient analysis in Question 3.1, this experiment performs a systematic ablation (“switch-off”) study to quantify the impact of key training components on a deep FCNN trained on Tiny ImageNet. Each experiment disables or modifies a single factor while keeping all other hyperparameters fixed.

3.2.2 Baseline Configuration

The reference model configuration was:

- **Activation:** Leaky ReLU ($\alpha = 0.01$)
- **Batch Normalization:** Enabled
- **Dropout** [13]: 0.3 after each hidden layer
- **Learning Rate:** 0.001
- **Optimizer:** AdamW [14]
- **Epochs:** 20
- **Batch Size:** 64

All ablation experiments modify exactly one component relative to this baseline.

3.2.3 Ablation Experiments

Test 1: Removing Dropout **Expectation:** Increased overfitting due to the absence of regularization.

- Training accuracy increased while validation accuracy decreased.
- Training-validation gap widened over epochs.
- Clear signs of overfitting in later stages of training.

Test 2a: Learning Rate $\times 10$ Higher (0.01) **Expectation:** Faster learning with potential instability.

- Rapid initial loss reduction.
- Large oscillations in training and validation loss.
- Failure to converge to a stable optimum.

Test 2b: Learning Rate $\times 10$ Lower (0.0001) **Expectation:** Stable but slow convergence.

- Smooth but nearly flat loss curves.
- Underfitting within the limited training budget.
- Significantly lower final accuracy.

Test 3: AdamW $\rightarrow$ Vanilla SGD **Change:** Optimizer replaced with vanilla SGD (no momentum).

- Much slower convergence.
- Higher variance in loss curves.
- Substantially degraded final performance.

3.2.4 Quantitative Comparison

Final validation and test accuracies for all configurations are summarized in Table 3.

Table 3: Final Validation and Test Accuracy for Ablation Experiments

Configuration	Validation Accuracy (%)	Test Accuracy (%)
Baseline (AdamW + Dropout 0.3 + LR 0.001)	42.00	41.00
No Dropout	41.20	37.80
Learning Rate = 0.01	38.20	38.70
Learning Rate = 0.0001	33.60	34.70
Vanilla SGD	23.80	20.80

3.2.5 Training Dynamics

Figure 7 illustrates the training and validation accuracy curves for the baseline and ablation configurations.



Figure 7: Training and validation accuracy comparison for ablation experiments. The baseline configuration exhibits the most stable and highest performance, while vanilla SGD and poorly tuned learning rates significantly degrade performance.

3.2.6 Impact Analysis

The relative importance of the ablated components is summarized as follows:

- Optimizer:** AdamW has the largest impact, providing faster convergence and superior final accuracy due to adaptive learning rates and momentum.
- Learning Rate:** A $10\times$ deviation in either direction significantly harms performance, highlighting the sensitivity of deep FCNNs to learning rate selection.

3. **Dropout:** While its immediate impact is smaller, dropout plays a crucial role in controlling overfitting and improving generalization.

3.2.7 Conclusion

The ablation study confirms that optimizer choice is the most critical factor for training deep FCNNs on Tiny ImageNet, followed by careful learning rate tuning. Dropout serves as an essential regularization mechanism, particularly for longer training runs. The baseline configuration represents a balanced trade-off between stability, convergence speed, and generalization performance.

4 Overall Findings and Conclusions

This assignment explored the behavior of Fully Connected Neural Networks across a range of settings, from from-scratch implementations to vision tasks and robustness studies.

Key findings include:

- Feature scaling is essential for stable optimization and faster convergence in gradient-based learning.
- FCNNs can learn meaningful representations from image data despite lacking spatial inductive bias, but do so inefficiently.
- Deep FCNNs suffer severely from vanishing gradients when using Sigmoid activations, while modern techniques such as ReLU and Batch Normalization enable stable training.
- Optimizer choice and learning rate selection have a greater impact on performance than architectural depth alone.

Overall, the experiments highlight both the expressive power and fundamental limitations of Fully Connected Neural Networks, motivating the use of convolutional architectures for scalable vision tasks.

5 References

References

- [1] Dua, D. and Graff, C. (2017). *UCI Machine Learning Repository: Adult Data Set*. University of California, Irvine, School of Information and Computer Sciences. <https://archive.ics.uci.edu/ml/datasets/adult>
- [2] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11):2278-2324. <http://yann.lecun.com/exdb/mnist/>
- [3] Le, Y. and Yang, X. (2015). *Tiny ImageNet Visual Recognition Challenge*. Stanford University CS231n. <http://cs231n.stanford.edu/tiny-imagenet-200.zip>
- [4] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org/>
- [5] He, K., Zhang, X., Ren, S., and Sun, J. (2015). *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pages 1026-1034. <https://arxiv.org/abs/1502.01852>

- [6] LeCun, Y., Bottou, L., Orr, G. B., and Müller, K. R. (2012). *Efficient BackProp*. In *Neural Networks: Tricks of the Trade*, pages 9-50. Springer. <http://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf>
- [7] Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch: An imperative style, high-performance deep learning library*. In *Advances in Neural Information Processing Systems* (NeurIPS), pages 8024-8035. <https://pytorch.org/>
- [8] Kingma, D. P. and Ba, J. (2014). *Adam: A method for stochastic optimization*. arXiv preprint arXiv:1412.6980. <https://arxiv.org/abs/1412.6980>
- [9] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). *Gradient-based learning applied to document recognition*. *Proceedings of the IEEE*, 86(11):2278-2324.
- [10] Hochreiter, S. (1998). *The vanishing gradient problem during learning recurrent neural nets and problem solutions*. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2):107-116. <https://ml.jku.at/publications/older/3504.pdf>
- [11] Maas, A. L., Hannun, A. Y., and Ng, A. Y. (2013). *Rectifier nonlinearities improve neural network acoustic models*. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 30(1). https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf
- [12] Ioffe, S. and Szegedy, C. (2015). *Batch normalization: Accelerating deep network training by reducing internal covariate shift*. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 448-456. <https://arxiv.org/abs/1502.03167>
- [13] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). *Dropout: A simple way to prevent neural networks from overfitting*. *Journal of Machine Learning Research*, 15(1):1929-1958. <http://jmlr.org/papers/v15/srivastava14a.html>
- [14] Loshchilov, I. and Hutter, F. (2017). *Decoupled weight decay regularization*. arXiv preprint arXiv:1711.05101. <https://arxiv.org/abs/1711.05101>